L3 MIS : Génie Logiciel

Implémentation du système de convoyeurs

Documents de départs

Scénario

On souhaite implémenter le système de convoyeurs selon l'architecture mise en place. Remarques : pour pouvoir effectuer les tests et visualiser, on va implémenter plus de choses que ce qui concerne strictement les convoyeurs.

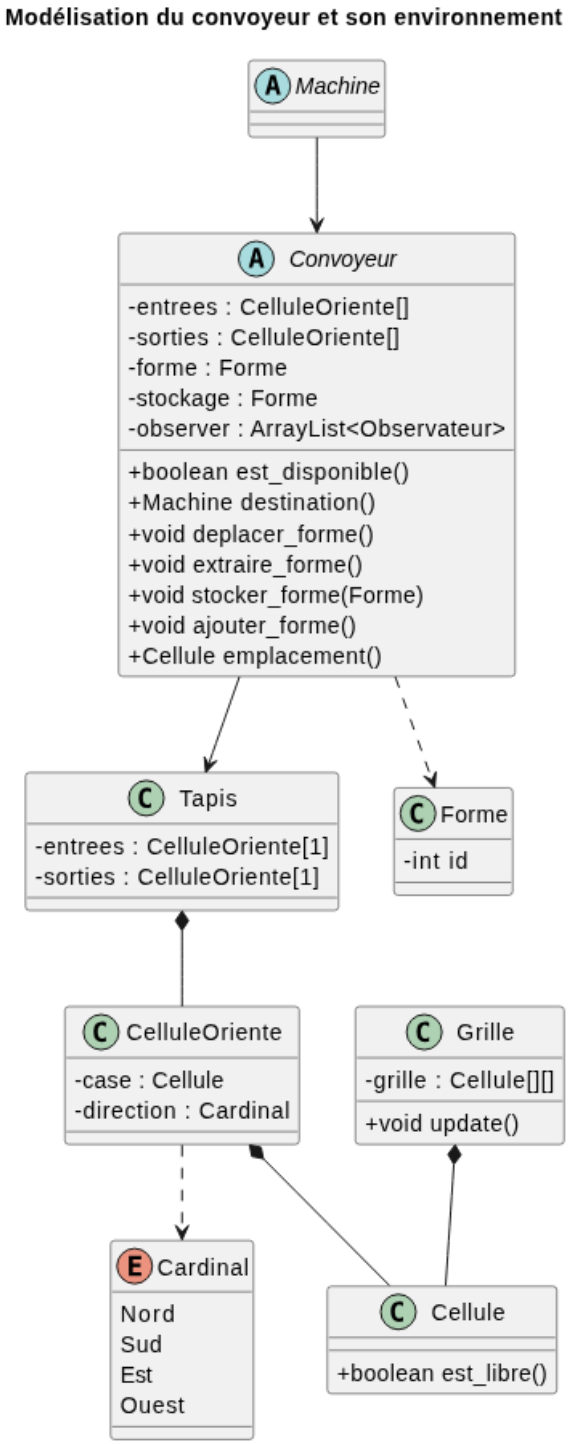
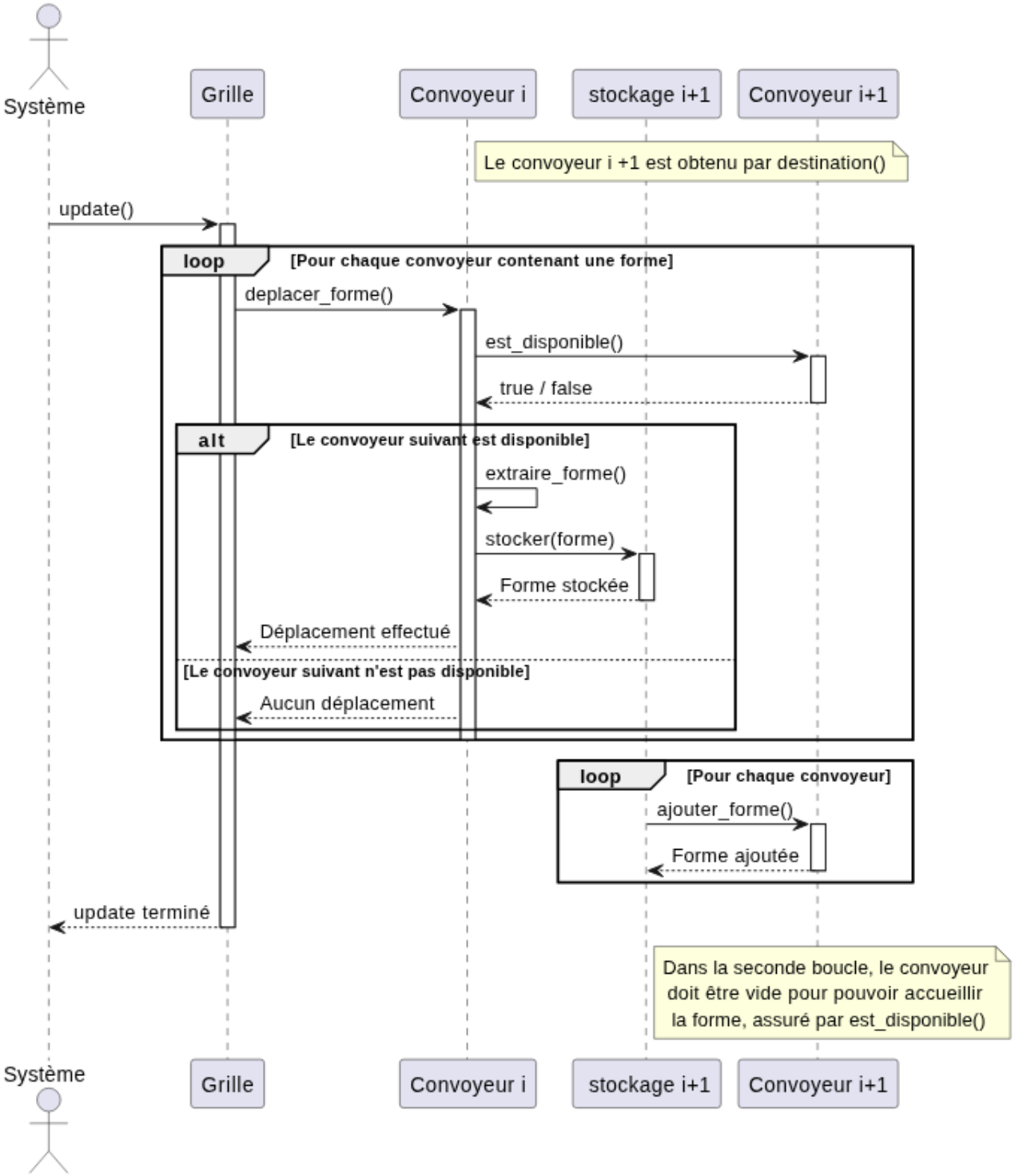


Illustration de la méthode update()



Commentaires :

- Les convoyeurs sont un type de machine, ils peuvent devoir envoyer leur forme à une machine qui n'est pas un convoyeur.
- La méthode estdisponible() d'un convoyeur renvoie true si le convoyeur ne possède pas de forme ou si la machine suivante est disponible (appels récursifs).
- La méthode update() de grille est détaillée par un diagramme, les implémentations des méthodes sur le convoyeur doivent coïncider avec ces spécifications.
- La méthode destination() ne renvoie une machine que si les directions d'entrée et de sortie sont compatibles.
- La méthode estLibre() de Cellule revoie false si la Cellule contient déjà une Machine.
- Les getters/setters ont été omis sur le diagramme, vous pouvez, et même devez, les rajouter lorsque c'est nécessaire.

> Pour l'implémentation et les tests, il est possible d'implémenter des classes qui ne seront pas dans le projet final, par exemple une classe forme qui permette juste de vérifier que les convoyeurs déplacent bien les objets.

Scénarios de test à prévoir

Les tests devront notamment couvrir les situations suivantes :

1. Réception d'une forme par un convoyeur vide.
2. Refus de réception d'une forme par un convoyeur qui ne peut pas l'accepter.
3. Transfert d'une forme entre deux convoyeurs compatibles.
4. Absence de transfert lorsque la destination est occupée.
5. Reprise du transfert lorsque la destination redevient disponible.
6. Transport successif de plusieurs formes sur une même ligne de convoyeurs.
7. Changement de direction d'un convoyeur.
8. Gestion d'un convoyeur sans débouché.
9. Détection d'un conflit lorsque plusieurs transferts visent une même destination.
10. Vérification de l'absence de perte ou de duplication de formes après plusieurs mises à jour.
11. Vérification du comportement de la simulation indépendamment de l'ordre de parcours des convoyeurs.

Sujet

Les scénarios de tests peuvent être à adapter si vous n'avez pas utilisé la même architecture. Rappel : vous ne pouvez pas implémentez une méthode et les tests associés, unitaires ou scénarios de tests

1. Créer le dossier du projet et les fichiers pour les différentes classes. Inclure un dossier de tests, il y aura au minimum un fichier de test par classes.
2. Écrire le scénario de test n°1 : un convoyeur vide possède une forme dans le stockage et ajouterforme() est appelée. Réussite : l'attribut forme contient la Forme et l'attribut stockage est vide.
3. Écrire le scénario de test n°1bis : la méthode stockerforme est appelée sur un convoyeur vide avec un paramètre Forme. Réussite : l'attribut stockage contient la Forme.

4. Écrire le scénario de test n°2 : un convoyeur possède une forme1 dans le stockage et stocker-forme() est appelée. Réussite : l'attribut stockage contient la forme1
5. Écrire le scénario de test n°2bis : un convoyeur possède une forme1 dans l'attribut forme une forme2 dans l'attribut stockage. ajouterforme() est appelée. Réussite : l'attribut forme contient la forme 1 et l'attribut stockage contient la forme2.
6. Écrire le scénario de test n°3, vérification de destination() : Deux convoyeurs placés successivement (selon les règles établies), le premier contient un Forme dans l'attribut forme et le second est vide. Appel de délpacerforme() sur le convoyeur 1. Réussite :le premier convoyeur est vide et le second contient une forme dans l'attribut forme.
7. Écrire le scénario de test n°4 : Deux convoyeurs placés successivement (selon les règles établies), le premier contient une Forme forme1 dans l'attribut forme et le second une Forme forme2 dans l'attribut Forme. Appel de délpacerforme() sur le convoyeur 1. Réussite : le premier contient une Forme forme1 dans l'attribut forme et le second une Forme forme2 dans l'attribut Forme.
8. Écrire le scénario de test n°5 : Deux convoyeurs placés successivement (selon les règles établies), le premier contient une Forme forme1 dans l'attribut forme et le second une Forme forme2 dans l'attribut Forme. Appel de délpacerforme() sur le convoyeur 1. Suppression de la forme sur le convoyeur 2, appel de déplacerforme() sur le convoyeur 1. Réussite : le premier convoyeur est vide et le second contient une Forme forme1 dans l'attribut Forme.
9. Écrire le scénario de test n°6 : (débrouillez-vous c'est facile et long à décrire)
10. Écrire le scénario de test n°7 : 4 convoyeur C1 C2 C3 C4 sur les cases (0,0), (0,1), (1,0), (1,1) C1 orienté nord-sud, C2 orienté est-sud, C3 et C4 orientés est-ouest. Réussite : C1.destination = C3, C2.destination = null (note : direction incompatible), C3.destination = C4, C4.destination = null.
11. Écrire le scénario de test n°7bis : 4 convoyeur C1 C2 C3 C4 sur les cases (0,0), (0,1), (1,0), (1,1) C1 orienté nord-sud, C2 orienté est-sud, C3 et C4 orientés est-ouest. La sortie de C1 est modifiée en est. Réussite : C1.destination =C2.
12. Écrire le scénario de test n°8 : Convoyeur C tel que C.destination = null contenant une Forme dans l'attribut forme. Appel de deplacerforme() sur C. Réussite : C contient une Forme dans l'attribut forme.
13. Écrire le scénario de test n°9 : Trois convoyeurs C1,C2,C3 est (0,0) (1,0) (2,0). C1 orienté est-ouest, C2 orienté est-sud,C3 orienté, ouest-est. Réussite C1.destination =C2, C3.destination()=null.
14. Écrire le scénario de test n°9bis : Trois convoyeurs C1,C2,C3 est (0,0) (1,0) (2,0). C1 orienté est-ouest, C2 orienté est-sud,C3 orienté, ouest-est. C1 contient une Forme f1 dans l'attribut forme et C3 contient une forme f3 dans l'attribut forme. Appel de update(). Réussite : C2 contient une Forme f1 dans l'attribut forme et C3 contient une forme f3 dans l'attribut forme.
15. Écrire le scénario de test n°10 : (Je détaille pas trop) : Créer un schéma de plusieurs convoyeurs avec différentes orientations et possédant ou non de forme. Lister les formes présentes. Appel de update() plusieurs fois. Réussite : La liste des formes présentes sur la grille est la même.
    Les autres tests sont des tests "plus grands" : tests d'intégration lors de la connexion à un autre module, tests fonctionnels dont on a testé une partie (ex F-008 à F012). On ne met pas en place certains tests (I003 par exemple) par soucis de simplicité.
16. Écrire la classe Forme (juste un constructeur à faire).
17. Écrire les tests unitaires de la classe Forme. (construction correcte de l'objet)
18. Écrire la classe CelluleOriente (juste un constructeur et l'enum à faire).
19. Écrire les tests unitaires de la classe CelluleOriente. (construction correcte de l'objet)
20. Écrire la classe Cellule.
21. Écrire les tests unitaires de la classe Cellule. (construction correcte de l'objet, vérification de estlibre())

22. Écrire la classe Tapis, en particulier un constructeur prenant en paramètres une cellule (vide), une direction d'entrée, une direction de sortie et créant le tapis associé.
23. Écrire les tests unitaires de la classe Tapis. (construction correcte de l'objet)
24. Écrire la méthode update() de la classe Grille.
25. Écrire le reste de la classe Grille.
26. Écrire les tests unitaires de la classe Grille.
    Pour la classe convoyeur, les tests des méthodes ont majoritairement déjà été fait mais il en reste quelques uns...
27. Écrire la méthode estdisponible() de la classe Convoyeur.
28. Écrire la méthode destination() de la classe Convoyeur.
29. Écrire la méthode deplacerforme() de la classe Convoyeur.
30. Écrire la méthode extraireforme() de la classe Convoyeur.
31. Écrire la méthode stockerforme() de la classe Convoyeur.
32. Écrire la méthode ajouterforme() de la classe Convoyeur.
33. Écrire la méthode emplacement() de la classe Convoyeur.
34. Écrire le test de la méthode emplacement() de la classe Convoyeur
35. Écrire le test de la méthode estdisponible() de la classe Convoyeur